

EXERCISE MPQT-01

A qubit simulator in NumPy

Every postulate in Unit 3 is a line or two of NumPy. Write those lines yourself and you have a working simulator for one and two qubits.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

EXERCISE

mpqt-ex01-qubit-simulator

FORMAT

One Python file

SUBMIT AS

<ROLLNO>.py

TOPIC

States, Born rule, gates, measurement, tensor products, expectation values

MARKS

14 (part of Part C, 20)

OFFERING

Odd Semester 2026-27

What you are building

A single file, `qubit.py`, that holds kets as NumPy arrays and does to them what the postulates say: normalise them, take inner products, compute Born probabilities, apply gates, measure and collapse, combine qubits with the tensor product, and compute expectation values.

This is the coding half of **Part C of the class assignment** (The Qubit Workbook). It carries 14 of Part C's 20 marks. The other 6 are for a short report in which you use this module to run three experiments; the assignment sheet describes them. So the module has to work, and you have to be able to use it.

The starter file is in `starter/qubit.py`. Every function in it raises `NotImplementedError` and several constants are `None`. Replace each one.

Conventions

These are fixed. The tests assume them, and so does everything in the notes.

- A **ket** is a one-dimensional NumPy array of complex numbers, shape `(n,)`. So $|0\rangle$ is `np.array([1, 0], dtype=complex)`. A ket is a column vector on paper; NumPy does not need the distinction, and 1-D arrays keep the code short.
- A **basis** is a Python list of kets that are orthonormal, for example `[KET0, KET1]` or `[PLUS, MINUS]`.
- A **gate** or an **observable** is a two-dimensional array, shape `(n, n)`.
- Two-qubit kets are ordered $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ as entries 0 to 3 of the array. The leftmost symbol is the first qubit, so $|10\rangle = |1\rangle \otimes |0\rangle$.
- Randomness comes **only** from the `rng` argument, a `numpy.random.Generator` made with `np.random.default_rng(seed)`. The same seed must give the same run.

The constants

`KET0`, `KET1` and the 2×2 identity `I2` are given. You write the rest:

- `PLUS` and `MINUS`, the kets $|\pm\rangle = \frac{1}{\sqrt{2}}(|0\rangle \pm |1\rangle)$;
- `X`, `Y` and `Z`, the Pauli matrices;
- `H`, the Hadamard gate, including its $\frac{1}{\sqrt{2}}$;
- `S`, the phase gate $\text{diag}(1, i)$;
- `CNOT`, the 4×4 controlled-NOT with the first qubit as control.

Writing a gate down from its definition is the check that you know it; the sign in Y and the $\frac{1}{\sqrt{2}}$ in H are where people slip. The tests of your *functions* use the checker's own copies of these matrices, so a slip in one constant costs only the stage that checks that constant.

The functions

Each has a docstring in the starter with a one-line example. Here is what each must do.

Stage 1: states and normalisation (2 marks)

```
def ket(*amplitudes) -> np.ndarray:
    """Build a ket from its amplitudes, as a complex 1-D array.
    Example: ket(1, 1j) -> array([1.+0.j, 0.+1.j])"""

def is_normalised(psi, tol=1e-9) -> bool:
    """True if the norm of psi is 1, to within tol.
    Example: is_normalised(ket(1, 1)) -> False"""

def normalise(psi) -> np.ndarray:
    """Return psi divided by its norm. Raise ValueError for the zero vector.
    Example: normalise(ket(1, 1)) -> array([0.7071+0.j, 0.7071+0.j])"""
```

`ket` must work for any number of amplitudes, and `normalise` must return a new array without changing the one you passed in. Also checked here: `PLUS` and `MINUS`.

Stage 2: inner product and Born probabilities (3 marks)

```
def inner(phi, psi) -> complex:
    """The inner product <phi|psi>, conjugating the FIRST argument.
    Example: inner(ket(1j, 0), ket(1, 0)) -> -1j"""

def probabilities(psi, basis) -> np.ndarray:
    """Born-rule probabilities p_k = |<b_k|psi>|^2, one per basis ket.
    Return real floats. Raise ValueError if psi is not normalised.
    Example: probabilities(PLUS, [KET0, KET1]) -> array([0.5, 0.5])"""
```

PITFALL · CONJUGATE THE FIRST ARGUMENT

In this course $\langle \phi | \psi \rangle = \sum_k \phi_k^* \psi_k$. The star is on ϕ , the first argument. Many maths texts, and some NumPy functions, put it on the second. Conjugating the wrong one gives the complex conjugate of the right answer, and forgetting it altogether lets a nonzero vector such as $(1, i)$ have "length" zero. There is a test for exactly this.

A probability is a real number. $|z|^2$ computed as `abs(z)**2` is real; computed as `z * np.conj(z)` it is a complex number with zero imaginary part, and the test rejects that.

Stage 3: gates, unitarity and Hermiticity (3 marks)

```
def is_unitary(U, tol=1e-9) -> bool:
    """True if U is square and U^dagger U = I, entry by entry to within tol.
    Example: is_unitary(H) -> True"""

def is_hermitian(A, tol=1e-9) -> bool:
    """True if A is square and A^dagger = A, entry by entry to within tol.
    Example: is_hermitian(Y) -> True"""

def apply(U, psi) -> np.ndarray:
    """Return the ket U|psi>. Raise ValueError if the sizes do not match.
    Example: apply(X, KET0) -> array([0.+0.j, 1.+0.j])"""
```

A matrix that is not square is neither unitary nor Hermitian: return `False`, do not crash. The dagger is the **conjugate** transpose, `A.conj().T`. A test uses a matrix that equals its plain transpose but is not Hermitian. Also checked here: `X`, `Y`, `Z`, `H`, `S`, and the identities $HH = I$ and $HZH = X$ with your own `H` and `Z`.

Stage 4: measurement and collapse (2 marks)

```
def measure(psi, basis, rng):
    """Measure psi in the given orthonormal basis. Return (k, post):
    k is the outcome index, drawn with the Born probabilities, and post
    is basis[k], the state after collapse.
    Example: measure(KET1, [KET0, KET1], rng) -> (1, array([0.+0.j, 1.+0.j]))"""
```

Draw the outcome with the generator you are given, for instance `rng.choice(len(basis), p=p)`. The tests measure thousands of times and compare the frequencies with the Born rule, with generous bounds, so a correct implementation passes whatever way you draw. They also check that a certain outcome is always what happens, that the same seed repeats the same run, and that the ket you measured is left unchanged.

PITFALL · PYTHON COUNTS FROM 0

The notes number vector entries from 1, so v_1 is the first entry. NumPy numbers them from 0, so the same entry is `v[0]`. Outcome index `k` in this exercise is a NumPy index: measuring $|1\rangle$ in `[KET0, KET1]` gives `k = 1`.

Stage 5: tensor products (2 marks)

```
def tensor(*items) -> np.ndarray:
    """Kronecker product of kets or of matrices, taken left to right.
    Example: tensor(KET0, KET1) -> array([0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j])"""
```

`tensor(a, b, c)` is $a \otimes b \otimes c$, and the order matters: `tensor(X, I2)` flips the first qubit, `tensor(I2, X)` the second. `np.kron` does one product of two things; you write the loop. Also checked here: `CNOT`, and that $\text{CNOT}(H \otimes I) |00\rangle = |\Phi^+\rangle$.

Stage 6: expectation values (2 marks)

```
def expectation(A, psi) -> float:
    """The expectation value <psi|A|psi> of a Hermitian A, as a real float.
    Raise ValueError if A is not Hermitian.
    Example: expectation(Z, PLUS) -> 0.0"""
```

When A is Hermitian the number $\langle \psi | A | \psi \rangle$ is real, but NumPy computes it as a complex number with a tiny or zero imaginary part. Return the real part as a Python `float`.

Rules

- **NumPy and the Python standard maths modules only** (`numpy`, `math`, `cmath`). No qiskit, no other quantum library, no SciPy. The checker reads your imports and fails every stage if it finds anything else.
- Keep the function names and arguments exactly as in the starter. The grader imports your file and calls them by name.
- Importing your file must not do any work. Put your Part C experiments, plots and prints under `if __name__ == "__main__":` at the bottom. A file that crashes or prints on import fails.
- Individual work. You may discuss ideas; the code you submit is yours, and in the viva you will be asked to explain any line of it.

Checking your work

From the exercise folder:

```
./selfcheck.sh          # checks starter/qubit.py
./selfcheck.sh AIE25001.py # checks your own file
```

It runs the six stages with the public tests and prints one line per test: `ok`, or `FAIL:` followed by what was expected and what your code gave. On Google Colab or Windows, where the shell script may not run, call the tests directly, one stage at a time:

```
python3 tests/public.py qubit.py --stage states
```

The stages are `states`, `probabilities`, `gates`, `measure`, `tensor` and `expectation`. Do them in that order. Later functions may call earlier ones, and your `measure` will almost certainly call your `probabilities`.

NOTE · PUBLIC AND HIDDEN TESTS

The grader runs the public tests plus hidden tests on the same functions: other sizes of ket, random states and bases, three-outcome measurements. A clean self-check is necessary but not sufficient. If your code follows the docstrings for inputs of any size, three- and four-entry kets included, the hidden tests hold no surprises.

How this is marked

STAGE	MARKS	WHAT IT CHECKS	TAUGHT IN
States and normalisation	2	<code>ket</code> , <code>is_normalised</code> , <code>normalise</code> , <code>PLUS</code> , <code>MINUS</code>	Weeks 6, 11
Inner product and Born rule	3	<code>inner</code> with the right conjugation, <code>probabilities</code> in several bases	Weeks 6, 12
Gates, unitarity and Hermiticity	3	<code>is_unitary</code> , <code>is_hermitian</code> , <code>apply</code> , the five gates	Weeks 8, 13
Measurement and collapse	2	<code>measure</code> : frequencies, collapse, reproducibility	Week 12
Tensor products and CNOT	2	<code>tensor</code> for kets and matrices, <code>CNOT</code> , the Bell state	Weeks 9, 14
Expectation values	2	<code>expectation</code> , real output, refusal of non-Hermitian input	Week 15

A stage earns its marks when every test in it, public and hidden, passes. There is no partial credit inside a stage, so read each `FAIL:` line and fix it rather than moving on.

Submitting

One file, named for your roll number:

```
AIE25001.py
```

It is due at the end of week 15, with the Part C report. The assignment sheet has the submission channel and the late policy.